МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ "ЛЬВІВСЬКА ПОЛІТЕХНІКА"

Інститут **КНІТ**
Кафедра **ПЗ**

ЗВІТ

До лабораторної роботи № 2

З дисципліни: *"Опрацювання зображень методами штучного інтелекту"*

На тему: *"Розпізнавання дефектів фігур нейронною мережею. Частина 2. Алгоритм"*

Лектор:
доц. каф. ПЗ
Мельник Р.А.

Виконав:
ст. гр. ПЗІП-11
Лєсніков М.А.

Прийняв:
асистент кафедри ПЗ
Романюк О.І.

« ____ » _____ 2026 р.

∑=_____

Львів – 2026

ЗМІСТ

ЗАВДАННЯ 3

ХІД ВИКОНАННЯ 3

1. Опис алгоритму 4

1.1. Вхідні та вихідні дані 4

1.2. Етапи алгоритму 5

1.3. Різницеве зображення та кумулятивна гістограма 6

1.4. Нейромережа та навчання 7

1.5. Розпізнавання окремого зображення 8

2. Основні функції реалізації 8

3. Обґрунтування вибору алгоритму 13

4. Вибір технологій 15

5. Недоліки та обмеження реалізації 15

6. Аналіз можливості паралельного виконання 16

7. Паралельність залежно від обраної технології 17

8. Практичний приклад паралельного виконання 18

ВИСНОВКИ 20

ДОДАТОК А. КОД АЛГОРИТМУ 21

Тема роботи: Розпізнавання дефектів фігур нейронною мережею. Частина 2. Алгоритм

Мета роботи: Описати та обґрунтувати алгоритмічне рішення задачі розпізнавання дефектів фігур, проаналізувати його реалізацію, використані технології, обмеження та можливості оптимізації, зокрема за допомогою паралельного виконання.

ЗАВДАННЯ

Розробити застосунок розпізнавання зображень на JavaScript з використанням Brain.js і описати алгоритм його роботи.

Індивідуальний варіант: розробити застосунок для розпізнавання зображень суцільної фігури (круг, квадрат тощо) та цієї ж фігури із зайвою малою висячою поверхнею з використанням методу класифікації за кумулятивною гістограмою та нейронної мережі Brain.js.

ХІД ВИКОНАННЯ

Алгоритм реалізовано в браузері мовою JavaScript. Підготовка даних виконується пулом Web Workers, а навчання нейромережі Brain.js та розпізнавання виконуються в окремому воркері, тому інтерфейс залишається чутливим під час обчислень. Нижче алгоритм описано кроками, наведено ключові функції та проаналізовано можливості розпаралелення.

1. Опис алгоритму

1.1. Вхідні та вихідні дані

Вхідними даними алгоритму є зображення суцільної фігури на однотонному фоні у форматі PNG, JPG, BMP або WebP довільної роздільності. Під час навчання таке зображення створюється програмно: для нього генерується випадкова специфікація фігури (нормальна або з виступами), вона малюється на полотні та кодується у файл вибраного формату.

Результатом алгоритму є клас зображення ("без дефекту" або "дефект"), значення двох виходів мережі (впевненість), кумулятивна гістограма для графіка та час окремих етапів. Параметри, що впливають на роботу алгоритму, наведено в табл. 2.1.

Табл. 2.1

Параметри алгоритму

Параметр	Значення за замовчуванням	Вплив на алгоритм
Розмір робочого зображення	64×64 пікселі	Усі зображення зводяться до цього розміру, тому ознаки не залежать від роздільності файлу
Кількість кошиків гістограми	32	Крок кошика 256 / 32 = 8 рівнів яскравості; кількість входів мережі 2 × (32 - 1) = 62
Допуск	8 рівнів (нульовий кошик)	Різниця 0-7 вважається шумом (стиснення JPG, згладжування країв) і не враховується
Еталон	Круг по центру, радіус 30%	Зображення без дефектів, яке віднімається від зразка
Кількість зображень	3 групи по 300, разом 900	Розмір навчального набору (групи 32×32, 128×128, 1024×1024)
Формат файлів	Змішані PNG, JPG, BMP	JPG кодується з якістю 0,9; різні формати перевіряють стійкість до стиснення
Поділ вибірки	80% навчальна, 20% тестова	Перемішування з фіксованим seed 42
Кількість епох	2000	Навчання зупиняється раніше, якщо помилка не перевищує 0,005

Параметр	Значення за замовчуванням	Вплив на алгоритм
Швидкість навчання	0,3	Розмір кроку зміни ваг мережі
Воркери підготовки	До 8 (не більше ядер)	Кількість паралельних потоків підготовки зображень; порція 6 зображень

1.2. Етапи алгоритму

Алгоритм складається зі спільного конвеєра ознак і двох гілок: навчання та розпізнавання (рис. 2.1). Конвеєр ознак однаковий для обох гілок, завдяки чому розподіл ознак під час навчання збігається з розподілом під час розпізнавання.

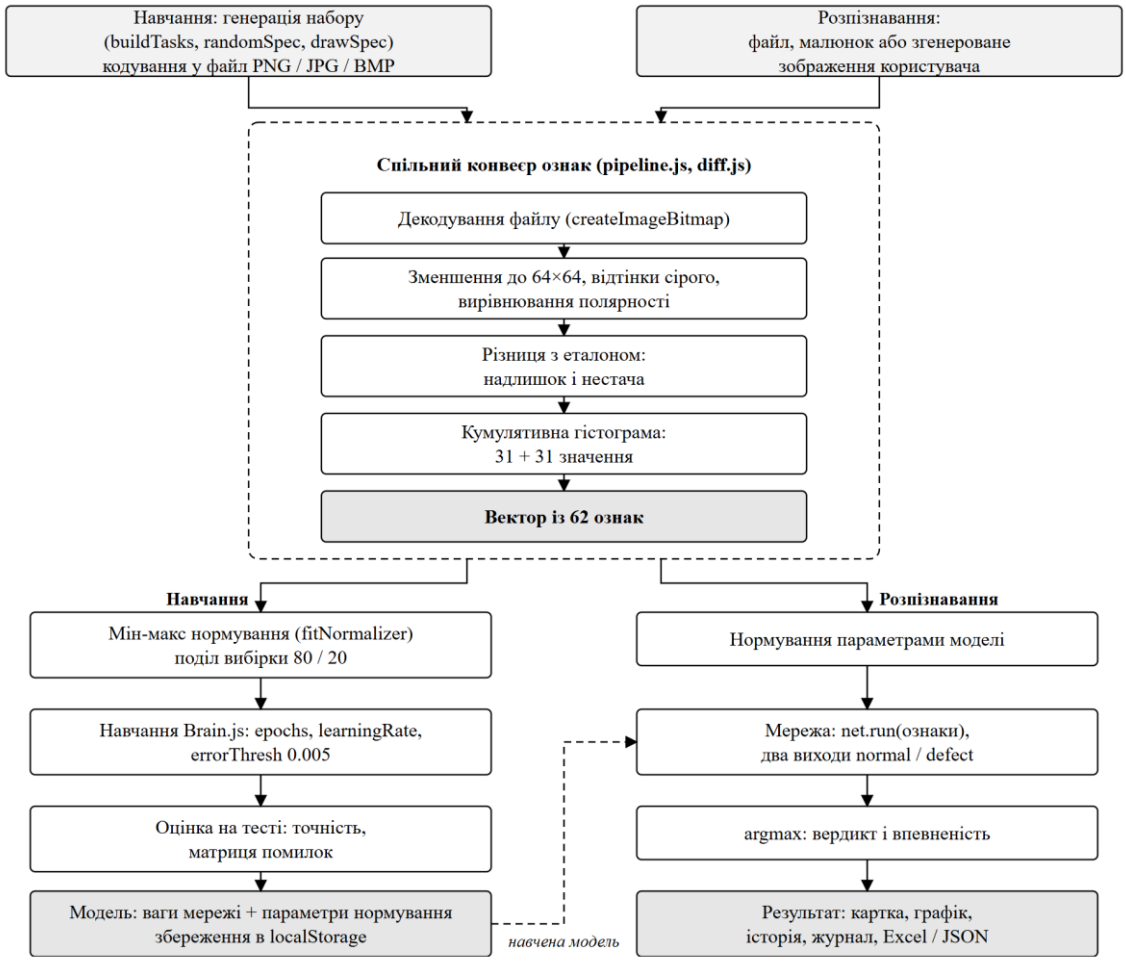


Рис. 2.1. Загальна схема алгоритму

Конвеєр ознак виконує такі кроки:

1. **Декодування.** Файл перетворюється на растрове зображення функцією createImageBitmap.
2. **Зведення до 64x64.** Зображення зменшується з вибраною якістю згладжування, переводиться у відтінки сірого за формулою 0,299 R + 0,587 G + 0,114 B. Далі вирівнюється полярність: якщо середня яскравість по рамці зображення більша за 127, зображення інвертується, щоб фон завжди був темним, а фігура світлою.
3. **Крок 3.** Різниця з еталоном. Від кожного пікселя зразка віднімається піксель еталона. Додатні значення утворюють канал "надлишок" (виступ), від'ємні, взяті за модулем, канал "нестача".
4. **Крок 4.** Кумулятивна гістограма. Для кожного каналу рахується гістограма з 32 кошиків по 8 рівнів, нульовий кошик (допуск) відкидається, решту 31 значення накопичуються й діляться на кількість пікселів 4096. Два канали дають 62 числа.

Гілка навчання продовжується так: ознаки нормуються мінімумом і максимумом кожного входу за навчальною вибіркою, вибірка ділиться 80 / 20, мережа Brain.js навчається, після чого перевіряється на тестових прикладах. Навчену мережу разом із параметрами нормування зберігають у браузері. Гілка розпізнавання нормує ознаки параметрами обраної моделі, запускає мережу та вибирає клас із більшим виходом.

1.3. Різницеве зображення та кумулятивна гістограма

Нехай I(x, y) яскравість пікселя зразка, E(x, y) яскравість еталона, d(x, y) = I(x, y) - E(x, y). Тоді канал надлишку дорівнює max(d, 0), а канал нестачі max(-d, 0). Для кожного каналу гістограма H[k] показує кількість пікселів, значення яких потрапило в кошик k (діапазон 8k ... 8k+7), а кумулятивна гістограма C[k] = (H[1] + ... + H[k]) / N, де N = 4096. На рис. 2.2 показано еталон, зразок із двома виступами та їхні різницеві зображення.



Рис. 2.2. Еталон, зразок з виступами та різницеві зображення

Для зразка без дефектів різниця з еталоном лежить у межах допуску, тому всі 62 ознаки дорівнюють нулю. Для зразка з виступами (рис. 2.3) значення кумулятивної гістограми каналу надлишку відрізняються від нуля; останнє значення дорівнює частці всіх пікселів, що відрізняються від еталона понад допуск. У наведеному прикладі це близько 2,1% (86 пікселів із 4096). Канал нестачі лишається нульовим, бо виступи лише додають яскравість.

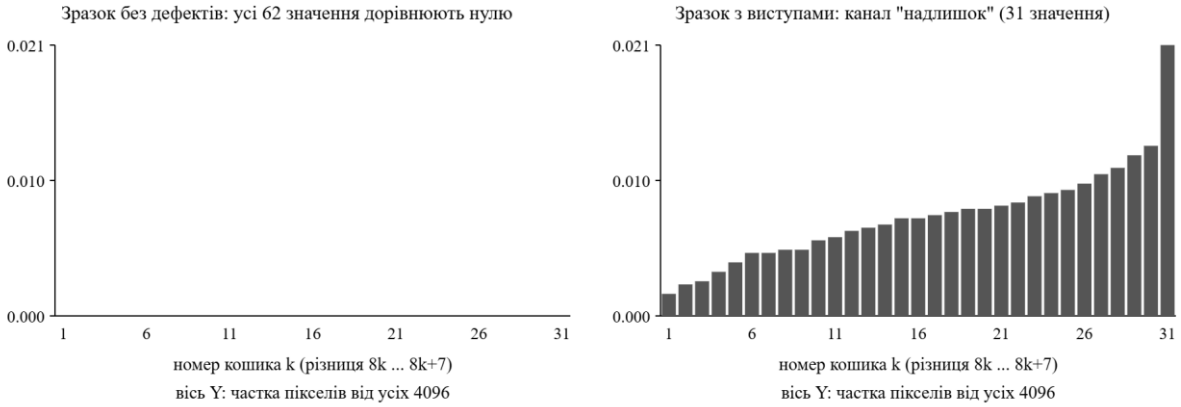


Рис. 2.3. Ознаки зразка без дефектів і зразка з виступами

1.4. Нейромережа та навчання

Мережа Brain.js (клас NeuralNetwork) має 62 входи та 2 виходи (normal і defect). Приховані шари відсутні, тому кожен вихід є сигмоїдою від зваженої суми входів плюс зсув.

Навчання виконується зворотним поширенням помилки: для кожного прикладу обчислюється відхилення виходів від правильної відповіді (1 для свого

класу, 0 для іншого), після чого ваги змінюються пропорційно швидкості навчання. Навчання триває задану кількість епох або до моменту, коли помилка не перевищує 0,005. Після навчання мережа перевіряється на тестовій вибірці, яку вона не бачила: рахуються точність, матриця помилок і точність за групами якості та форматами файлів.

1.5. Розпізнавання окремого зображення

Під час розпізнавання застосовується той самий конвеєр ознак, що й під час навчання, а потім ознаки нормуються параметрами моделі та подаються на вхід мережі. Вибирається клас із більшим виходом, його значення показується як впевненість. Застосунок також вимірює час окремих етапів: декодування і зменшення, обчислення гістограми та роботи мережі.

2. Основні функції реалізації

Основні обчислення зосереджено в кількох функціях (табл. 2.2). Для кожної вказано призначення, вхідні параметри, основні операції, результат і роль в алгоритмі. Повний код програми, що реалізує алгоритм, наведено в додатку А.

Табл. 2.2

Основні функції

Функція (файл)	Призначення та вхід	Основні операції та результат	Роль в алгоритмі
differenceImages (diff.js)	Різниця зразка й еталона. Вхід: два масиви яскравостей 64×64	Для кожного пікселя рахує різницю й розкладає її на надлишок і нестачу. Повертає два масиви	Крок 3 конвеєра
diffHistogramFeatures (diff.js)	Вектор ознак мережі. Вхід: зразок, еталон, кількість кошиків	Будує гістограму кожного каналу, відкидає нульовий кошик, накопичує й ділить на кількість пікселів. Повертає 62 числа	Крок 4

Функція (файл)	Призначення та вхід	Основні операції та результат	Роль в алгоритмі
decodeToGray, downscaleToGray (pipeline.js)	Файл у масив 64×64. Вхід: Blob або растр	Декодує файл, зменшує, переводить у відтінки сірого, вирівнює полярність. Повертає масив яскравостей	Кроки 1-2 конвеєра
prepareSample (pipeline.js)	Один навчальний приклад. Вхід: параметри завдання (група, формат, seed)	Генерує специфікацію, малює, кодує у файл, декодує, рахує ознаки. Повертає клас, групу, формат і ознаки	Одиниця роботи для пулу воркерів
prepareDataset (pool.js)	Підготовка всього набору. Вхід: завдання, кількість воркерів, колбек прогресу	Ділить завдання на порції, роздає воркерам із динамічної черги, збирає й сортує результати	Розпаралелення підготовки
fitNormalizer, applyNormalizer, splitSamples (dataset.js)	Підготовка вибірки. Вхід: масив зразків	Рахує мінімум і максимум кожної ознаки, нормує її до 0-1, перемішує та ділить вибірку 80 / 20	Підготовка входів мережі
train (train.worker.js)	Навчання. Вхід: зразки, конфігурація	Створює мережу, навчає, рахує точність і матрицю помилок. Повертає метрики та вагу моделі	Навчання і оцінка
recognize (train.worker.js)	Розпізнавання. Вхід: файл, модель	Декодує, рахує ознаки, нормує, запускає мережу. Повертає клас, виходи, гістограму й час етапів	Гілка розпізнавання

Нижче наведено фрагменти коду основних функцій.

Лістинг 2.1. Різницеве зображення (diff.js)

```
export function differenceImages(gray, reference) {
  const excess = new Uint8ClampedArray(gray.length);
  const deficit = new Uint8ClampedArray(gray.length);
  for (let i = 0; i < gray.length; i += 1) {
    const d = gray[i] - reference[i];
    if (d > 0) excess[i] = d;
    else deficit[i] = -d;
  }
  return { excess, deficit };
}
```

Лістинг 2.2. Кумулятивна гістограма та ознаки мережі (diff.js)

```
function cumulativeOfNonZero(diff, bins) {
  const counts = histogram(diff, bins);
  const out = [];
  let acc = 0;
  for (let k = 1; k < bins; k += 1) {
    acc += counts[k];
    out.push(acc / diff.length);
  }
  return out;
}

export function diffHistogramFeatures(gray, reference, bins) {
  const { excess, deficit } = differenceImages(gray, reference);
  return [...cumulativeOfNonZero(excess, bins), ...cumulativeOfNonZero(deficit,
bins)];
}
```

Лістинг 2.3. Декодування та зведення зображення до 64×64 (pipeline.js)

```
export function downscaleToGray(source) {
  const canvas = new OffscreenCanvas(TRAIN_SIZE, TRAIN_SIZE);
  const ctx = canvas.getContext('2d', { willReadFrequently: true });
  ctx.imageSmoothingEnabled = true;
  ctx.imageSmoothingQuality = 'high';
  ctx.drawImage(source, 0, 0, TRAIN_SIZE, TRAIN_SIZE);
  const rgba = ctx.getImageData(0, 0, TRAIN_SIZE, TRAIN_SIZE).data;
  return normalizePolarity(rgbaToGray(rgba), TRAIN_SIZE, TRAIN_SIZE);
}

export async function decodeToGray(blob) {
  const bitmap = await createImageBitmap(blob);
  const { width, height } = bitmap;
  const gray = downscaleToGray(bitmap);
  bitmap.close();
  return { gray, width, height };
}
```

Лістинг 2.4. Підготовка одного навчального зображення (pipeline.js)

```
export async function prepareSample({ id, index, group, format, variety, shapes,
bins, singleBias, seed }) {
  const rand = createRng(seed);
  const label = index % 2 === 0 ? 'normal' : 'defect';
  const spec = randomSpec(rand, { defective: label === 'defect', variety,
shapes, singleBias, aligned: true });

  const canvas = new OffscreenCanvas(group.size, group.size);
  drawSpec(canvas.getContext('2d'), spec, group.size);

  const fileFormat = format === MIXED_FORMAT ? FILE_FORMATS[id %
FILE_FORMATS.length].value : format;
  const blob = await encodeCanvas(canvas, fileFormat);
```

```
  const { gray } = await decodeToGray(blob);

  return {
    id,
    label,
    group: group.id,
    format: fileFormat,
    bytes: blob.size,
    features: computeFeatures(gray, { refType: shapes[0], bins }),
  };
```

Лістинг 2.5. Нормування ознак і поділ вибірки (dataset.js)

```
export function fitNormalizer(samples) {
  const n = samples[0].features.length;
  const min = new Array(n).fill(Infinity);
  const max = new Array(n).fill(-Infinity);
  for (const { features } of samples) {
    features.forEach((v, i) => {
      if (v < min[i]) min[i] = v;
      if (v > max[i]) max[i] = v;
    });
  }
  return { min, max };
}

export function applyNormalizer({ min, max }, features) {
  return features.map((v, i) => {
    const span = max[i] - min[i];
    if (span < 1e-9) return 0;
    return Math.min(1, Math.max(0, (v - min[i]) / span));
  });
}

export function splitSamples(samples, trainRatio = 0.8, seed = 42) {
  const rand = createRng(seed);
  const shuffled = samples.slice();
  for (let i = shuffled.length - 1; i > 0; i -= 1) {
    const j = Math.floor(rand() * (i + 1));
    [shuffled[i], shuffled[j]] = [shuffled[j], shuffled[i]];
  }
  const cut = Math.floor(shuffled.length * trainRatio);
  return { train: shuffled.slice(0, cut), test: shuffled.slice(cut) };
}
```

Лістинг 2.6. Навчання мережі у воркері (train.worker.js)

```
function train(id, { samples, config }) {
  const started = performance.now();
  const { train: trainSet, test: testSet } = splitSamples(samples);
  normalizer = { ...fitNormalizer(trainSet), mode: FEATURE_MODE, refType:
config.shapes[0] };
```

```
  const toData = (s) => ({ input: applyNormalizer(normalizer, s.features),
output: { [s.label]: 1 } });
  net = new NeuralNetwork({ hiddenLayers: config.hiddenNeurons > 0 ?
[config.hiddenNeurons] : [] });

  const stats = net.train(trainSet.map(toData), {
    iterations: config.epochs,
    errorThresh: 0.005,
    learningRate: config.learningRate,
    callbackPeriod: Math.max(1, Math.floor(config.epochs / 100)),
    callback: ({ iterations: epoch, error }) => reply(id, 'progress', { epoch,
epochs: config.epochs, error }),
  });

  const confusion = { normal: { normal: 0, defect: 0 }, defect: { normal: 0,
defect: 0 } };
  const byGroup = {};
  const byFormat = {};
  let correct = 0;
  for (const sample of testSet) {
    const predicted = argmax(net.run(applyNormalizer(normalizer,
sample.features)));
    const ok = predicted === sample.label;
    confusion[sample.label][predicted] += 1;
    tally(byGroup, sample.group, ok);
    tally(byFormat, sample.format, ok);
    if (ok) correct += 1;
  }

  reply(id, 'done', {
    result: {
      accuracy: correct / testSet.length,
      confusion,
      byGroup,
      byFormat,
      trainError: stats.error,
      epochs: stats.iterations,
      trainSize: trainSet.length,
      testSize: testSet.length,
      trainMs: performance.now() - started,
      netJson: net.toJSON(),
      normalizer,
    },
  });
}
```

Лістинг 2.7. Розпізнавання зображення у воркері (train.worker.js)

```
async function recognize(id, { blob, model }) {
  if (model) {
    net = new NeuralNetwork();
    net.fromJSON(model.netJson);
    normalizer = model.normalizer;
  }
  if (!net) throw new Error('Модель не завантажена. Навчіть або оберіть
модель.');
```

```
  const total = performance.now();
  const steps = {};
  let mark = total;
  const lap = (name) => {
    const now = performance.now();
    steps[name] = now - mark;
    mark = now;
  };

  reply(id, 'progress', STEP_DECODE);
  const { gray, width, height } = await decodeToGray(blob);
  lap('decode');

  reply(id, 'progress', STEP_FEATURES);
  const mode = normalizer.mode ?? 'full';
  const length = normalizer.min.length;
  let features;
  let cumulative;
  if (mode === FEATURE_MODE) {
    const bins = diffBinsFromLength(length);
    const refType = normalizer.refType ?? 'circle';
    features = computeFeatures(gray, { refType, bins });
    cumulative = displayCumulative(gray, { refType, bins });
  } else {
    const bins = legacyBins(length, mode);
    features = legacyFeatures(gray, bins, mode);
    cumulative = cumulativeHistogram(gray, bins);
  }
  lap('histogram');

  reply(id, 'progress', STEP_CLASSIFY);
  const scores = net.run(applyNormalizer(normalizer, features));
  const label = argmax(scores);
  lap('predict');

  reply(id, 'done', {
    result: {
      label,
      scores: { normal: scores.normal, defect: scores.defect },
      cumulative,
      width,
      height,
      steps,
      totalMs: performance.now() - total,
    },
  });
}
```

3. Обґрунтування вибору алгоритму

Для варіанту 8 вибрано перший спосіб класифікації за кумулятивною гістограмою різницевого зображення. Цей вибір зумовлений кількома причинами:

1. **Відповідність задачі.** Спосіб прямо вимагається варіантом і добре підходить для задачі "є виступ чи ні": виступ додає ненульові пікселі в різницевому зображенні, а нормальна фігура дає нулі.
2. **Малий розмір входу.** Замість 4096 пікселів мережа отримує 62 числа, тому навчання на 900 зображеннях займає кілька секунд і проходить у браузері.
3. **Незалежність від положення виступу.** Гістограма рахує кількість пікселів, а не їхні координати, тому виступ у будь-якому місці дає однакові ознаки.
4. **Простота і прозорість.** Ознаки мають зрозумілий зміст ("скільки пікселів відрізняється принаймні на стільки"), а однощарову мережу легко пояснити та відлагодити.
5. **Стійкість до формату.** Допуск у 8 рівнів поглинає дрібний шум стиснення, тому PNG, JPG і BMP дають схожі ознаки.

Альтернативні способи порівняно з обраним наведено в табл. 2.3.

Табл. 2.3

Порівняння з альтернативними способами

Критерій	**Кумулятивна гістограма**	**Інтенсивності клітинок (другий спосіб)**	**Згорткова мережа (CNN)**
Що бачить мережа	Кількість відхилень від еталона	Яскравість кожної клітинки зображення	Саме зображення, ознаки вивчає сама
Положення дефекту	Не враховується	Враховується	Враховується
Кількість входів	62	Кількість клітинок	Тисячі пікселів
Потрібен еталон	Так	Ні	Ні
Орієнтовний розмір набору	Сотні зображень	Сотні зображень	Тисячі зображень
Орієнтовний час навчання	Секунди	Секунди - хвилини	Хвилини - години

4. Вибір технологій

Технології вибрано так, щоб усе працювало безпосередньо в браузері без сервера (табл. 2.4).

Табл. 2.4

Використані технології

Параметр	Значення за замовчуванням	Вплив на алгоритм
JavaScript	Уся логіка застосунку	Мова рекомендована у завданні; працює в браузері без встановлення
Brain.js 1.6.1	Нейромережа	Вимога завдання; проста API, працює на CPU у воркері
Web Workers	Підготовка даних, навчання, розпізнавання	Не блокують інтерфейс і дають паралельність на кількох ядрах
OffscreenCanvas, createImageBitmap	Малювання, кодування й декодування зображень у воркері	Доступні у воркері, де немає DOM; швидше за роботу з пікселями вручну
React 19, Redux Toolkit	Інтерфейс і стан застосунку	Зручно оновлювати прогрес-бари та показувати результат; thunks зручні для асинхронних операцій
SCSS-модулі, Vite	Стилі та збірка	Вимоги до стеку; Vite підтримує воркери як ES-модулі
localStorage	Збереження навчених моделей	Дає змогу зберегти модель без сервера
Recharts, ExcelJS	Графіки та експорт результату	Готові бібліотеки; ExcelJS підвантажується лише за потреби

5. Недоліки та обмеження реалізації

Поточна реалізація має обмеження, пов’язані з самим методом і з організацією обчислень. Для кожного з них нижче описано, коли воно проявляється та як зменшити його вплив.

Чутливість до зсуву та розміру фігури. Метод порівнює піксель із пікселем у тих самих координатах, тому зсунута чи збільшена фігура створює велику різницю з еталоном; навіть збільшення радіуса на 5% вже розпізнається як дефект. Щоб зменшити вплив, можна додати попереднє вирівнювання: знайти

фігуру за порогом яскравості, змістити її в центр і масштабувати до розміру еталона.

Навчання лише на згенерованих зображеннях. Точність 100% отримано на синтетичних даних того самого типу, тому вона не показує якість на реальних фото. Варто додати набір реальних зображень і незалежну перевірку, а навчальну вибірку доповнити шумом, зсувом і зміною яскравості.

Один еталон. Еталон будується за першим типом фігури зі списку, тому одна модель навчається на одній фігурі. Для інших фігур потрібно навчати окрему модель або вирівнювати зображення до єдиної форми.

Фіксований допуск. Допуск у 7 рівнів яскравості може виявитись замалим, якщо JPG сильно стиснений або фон шумний. Допуск можна зробити параметром або попередньо згладжувати зображення.

Втрата дрібних деталей. Дуже малі виступи на зображеннях малої роздільності можуть зникнути під час зведення до 64×64. Вплив зменшить збільшення робочого розміру або обробка зображення частинами.

Навчання в одному потоці. Зі збільшенням набору та кількості епох час навчання зростає лінійно й стає вузьким місцем.

Генерація великих зображень. Малювання й кодування файлів 1024×1024, особливо у форматі BMP, дорогі за часом і пам’яттю: 900 зображень займають понад 300 МБ у файлах. Тут допоможе менша частка великих груп, відмова від проміжних файлів або кешування набору.

6. Аналіз можливості паралельного виконання

Не всі частини алгоритму можна виконувати паралельно, тому спершу розглянемо залежності між кроками.

Що можна розпаралелити. Підготовка кожного зображення повністю незалежна від інших. Генерація специфікації та малювання залежать лише від власного seed, кодування й декодування лише від власного файлу, а різниця з

еталоном, гістограма й ознаки лише від власного зображення. Усе це виконується у воркерах пулу. Така задача добре розпаралелюється, і вона займає значну частину часу. Порівняння з однопотоковим режимом із вимірами буде виконано в лабораторній 3.

Що виконується послідовно. Збір результатів і їх сортування потребують усіх відповідей воркерів, але виконуються швидко в головному потоці. Нормування та поділ вибірки потребують усього набору, бо рахуються мінімуми й максимуми ознак. Навчання Brain.js виконується епоха за епохою, кожна використовує ваги попередньої, а бібліотека не вміє ділити навчання мережі між потоками, тому воно лишається в одному воркері. Розпізнавання одного зображення займає мілісекунди, тому розпаралелювати його недоцільно.

7. Паралельність залежно від обраної технології

Застосунок виконується в браузері, тому для паралельності використано Web Workers. Це окремі потоки виконання, які мають власну пам’ять та цикл подій і обмінюються з головним потоком повідомленнями.

Яку проблему вирішують. Головний потік браузера один і відповідає як за виконання JavaScript, так і за малювання інтерфейсу. Довге обчислення в ньому блокує сторінку: кнопки не натискаються, а прогрес-бар не оновлюється.

Що винесено у воркери. У пул воркерів (prep.worker.js) винесено підготовку зображень: генерацію, кодування, декодування та обчислення ознак. В окремий воркер (train.worker.js) винесено навчання Brain.js і розпізнавання. У головному потоці лишаються тільки керування, передача повідомлень і оновлення інтерфейсу.

Особливості. У воркері немає DOM, тому використано OffscreenCanvas і createImageBitmap, а Brain.js підключено з файлу dist без модуля gpu.js. Воркери створюються як ES-модулі (параметр type: "module"), що підтримує Vite. Дані

між потоками копіюються, тому у повідомленнях передаються лише невеликі масиви ознак (62 числа на зображення), а не самі зображення.

Вплив на чутливість інтерфейсу. Завдяки воркерам прогрес-бар оновлюється під час навчання, а кнопки лишаються доступними. Відповіді воркерів надходять у головний потік як макрозадачі, а код після очікування результату виконується як мікрозадача, тому важкої роботи в головному потоці не лишається.

Інші механізми. Можна було б використати SharedArrayBuffer із Atomics для спільної пам'яті, але для цього потрібні спеціальні заголовки безпеки сервера GPU-навчання через WebGL або WebGPU потребує іншої бібліотеки.

8. Практичний приклад паралельного виконання

Паралельність реалізовано у функції prepareDataset (лістинг 2.10). Схему роботи показано на рис. 2.4.

Лістинг 2.10. Розподіл підготовки зображень між воркерами (pool.js)

```
export async function prepareDataset(tasks, { workers, onProgress }) {
  const results = [];
  let done = 0;
  const report = (count) => {
    done += count;
    onProgress?.(done, tasks.length);
  };

  if (workers === 0) {
    for (const part of chunked(tasks, CHUNK_SIZE)) {
      for (const task of part) results.push(await prepareSample(task));
      report(part.length);
      await yieldToUi();
    }
  } else {
    await ensurePool(workers);
    const queue = chunked(tasks, CHUNK_SIZE);
    let next = 0;
    await Promise.all(
      pool.slice(0, workers).map(async (worker) => {
        while (next < queue.length) {
          const part = queue[next];
          next += 1;
          results.push(...(await call(worker, { type: 'prepare', tasks: part
}())));
          report(part.length);
        }
```

```
      }),
    );
  }
  return results.sort((a, b) => a.id - b.id);
}
```

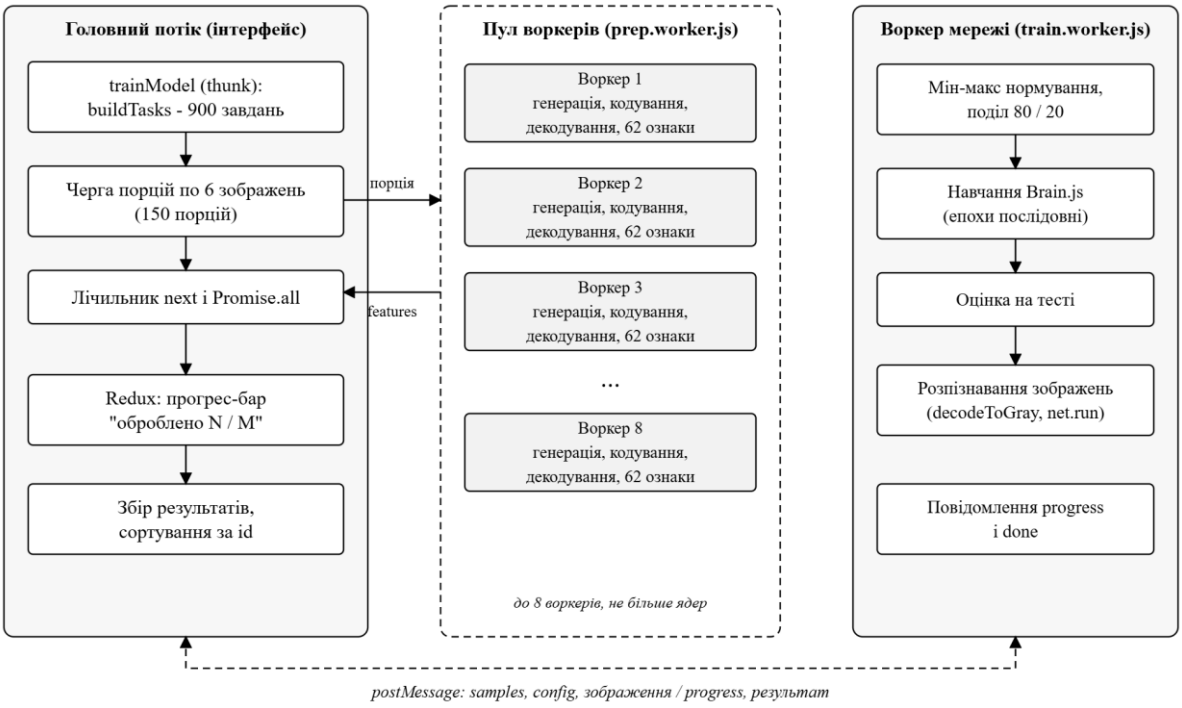


Рис. 2.4. Схема паралельної підготовки даних і роботи воркера мережі

Що виконується паралельно. Кожен воркер пулу виконує повний цикл підготовки для своєї порції з 6 зображень: генерує специфікацію, малює, кодує у файл, декодує та обчислює ознаки.

Як розподіляється робота. Список із 900 завдань ділиться на 150 порцій. Кожен воркер у циклі бере наступну порцію зі спільної черги (лічильник next) і після відповіді бере нову. Оскільки лічильник змінюється лише в головному потоці, гонки даних немає. Швидші воркери беруть більше порцій, тому навантаження вирівнюється автоматично.

Як збираються результати. Відповіді воркерів надходять як Promise. Функція Promise.all очікує, поки всі воркери завершать цикл, після чого результати сортуються за ідентифікатором, щоб порядок не залежав від

швидкості воркерів. Після кожної порції викликається колбек прогресу, тому прогрес-бар показує реальний хід "оброблено N з M".

Переваги. Використовуються кілька ядер процесора, інтерфейс не блокується, а розмір порції (6 зображень) дає баланс між частотою оновлення прогресу та витратами на передачу повідомлень. Якщо кількість воркерів дорівнює нулю, той самий код виконується в головному потоці з короткими паузами (setTimeout) для оновлення інтерфейсу; цей режим призначений для порівняння швидкості.

ВИСНОВКИ

Обраний алгоритм підходить для задачі, бо виступ на фігурі дає ненульову різницю з еталоном, а нормальна фігура дає нулі, тому 62 ознаки кумулятивної гістограми легко розділяються однощаровою мережею. Основні переваги: малий розмір входу, швидке навчання, незалежність від положення виступу і зрозумілі ознаки. Основні недоліки: чутливість до зсуву й розміру фігури, потреба в еталоні та навчання лише на згенерованих даних.

Потенційним вузьким місцем є послідовне навчання мережі в одному воркері, а також генерація й кодування великих зображень. Паралельне виконання для підготовки даних доцільне й використане: кожне зображення оброблюється незалежно, тому робота ділиться між ядрами. Для самого навчання паралельність у цій реалізації не виправдана.

Для значно більших обсягів даних я б додав попереднє вирівнювання фігур, кешування згенерованого набору, навчання міні-пакетами з використанням GPU та потоковий запис проміжних результатів, щоб не тримати весь набір у пам’яті.

ДОДАТОК А. КОД АЛГОРИТМУ

Нижче наведено код модулів, що реалізують алгоритм: генерацію зображень, ознаки, пул воркерів, навчання та розпізнавання. Код інтерфейсу у додатку не наведено.

A.1. src/shared/config/ml.js

```
export const TRAIN_SIZE = 64;

export const DATASET_GROUPS = [
  { id: 'icons', label: 'Іконки', size: 32 },
  { id: 'standard', label: 'Стандартні', size: 128 },
  { id: 'hq', label: 'Якісні', size: 1024 },
];

export const FILE_FORMATS = [
  { value: 'png', label: 'PNG' },
  { value: 'jpg', label: 'JPG' },
  { value: 'bmp', label: 'BMP' },
];

export const MIXED_FORMAT = 'mixed';

export const FEATURE_MODE = 'diffHist';

export const VARIETY_LEVELS = [
  { value: 'basic', label: 'Базова' },
  { value: 'extended', label: 'Розширена' },
];

export const VARIETY_HINT = 'Розширена: менші виступи різної форми.';

export const MAX_WORKERS = 8;

export const RECOGNITION_STEPS = [
  { stage: 'Декодування та масштабування до 64×64', value: 0.2 },
  { stage: 'Різниця з еталоном і ознаки', value: 0.7 },
  { stage: 'Класифікація мережею', value: 0.9 },
];
```

A.2. src/shared/lib/ml/rng.js

```
export function createRng(seed = 1) {
  let a = seed >>> 0;
  return function rand() {
    a = (a + 0x6d2b79f5) | 0;
    let t = Math.imul(a ^ (a >>> 15), 1 | a);
    t = (t + Math.imul(t ^ (t >>> 7), 61 | t)) ^ t;
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
  };
}

export const randRange = (rand, min, max) => min + rand() * (max - min);
```

A.3. src/shared/lib/ml/shapes.js

```
import { randRange } from './rng.js';

export const CLASSES = ['normal', 'defect'];
```

```
export const BASIC_TYPES = ['circle', 'square'];
export const EXTENDED_TYPES = [
  'circle',
  'ellipse',
  'square',
  'rectangle',
  'triangle',
  'hexagon',
];

export const SHAPE_LABELS = {
  circle: 'Круг',
  ellipse: 'Еліпс',
  square: 'Квадрат',
  rectangle: 'Прямокутник',
  triangle: 'Трикутник',
  hexagon: 'Шестикутник',
};

const SPIKE_START = 0.5;
const TWO_PI = Math.PI * 2;
const POLYGON_SIDES = { triangle: 3, hexagon: 6 };
const POLYGON_SCALE = { triangle: 1.3, hexagon: 1.05 };

function boundaryDistance(spec, theta) {
  const phi = theta - spec.rot;
  switch (spec.type) {
    case 'circle':
      return spec.r;
    case 'ellipse': {
      const a = spec.r;
      const b = spec.r * spec.aspect;
      return (a * b) / Math.hypot(b * Math.cos(phi), a * Math.sin(phi));
    }
    case 'square':
    case 'rectangle': {
      const a = spec.r;
      const b = spec.r * spec.aspect;
      return Math.min(
        a / Math.max(Math.abs(Math.cos(phi)), 1e-6),
        b / Math.max(Math.abs(Math.sin(phi)), 1e-6),
      );
    }
    default: {
      const n = POLYGON_SIDES[spec.type];
      const R = spec.r * POLYGON_SCALE[spec.type];
      const sector = TWO_PI / n;
      const local = (((phi % sector) + sector) % sector) - sector / 2;
      return (R * Math.cos(Math.PI / n)) / Math.cos(local);
    }
  }
}

const spikeEnd = (spec, spike) => boundaryDistance(spec, spike.angle) + spike.len;

export function nominalSpec(type) {
  return {
    type,
    cx: 0.5,
    cy: 0.5,
    r: 0.3,
    rot: 0,
    aspect: type === 'ellipse' || type === 'rectangle' ? 0.7 : 1,
    spikes: [],
  };
}
```

```
export function randomSpec(
  rand,
  {
    defective,
    type,
    shapes = BASIC_TYPES,
    variety = 'basic',
    singleBias = 0,
    aligned = false,
  } = {},
) {
  const extended =
    variety === 'extended' || (type !== undefined && !BASIC_TYPES.includes(type));
  const shape = type ?? shapes[Math.floor(rand() * shapes.length)];


  const spec = {
    type: shape,
    cx: randRange(rand, 0.44, 0.56),
    cy: randRange(rand, 0.44, 0.56),
    r: extended ? randRange(rand, 0.27, 0.33) : randRange(rand, 0.295, 0.305),
    rot: 0,
    aspect: 1,
    spikes: [],
  };
  if (extended) {
    if (shape !== 'circle') spec.rot = randRange(rand, 0, TWO_PI);
    if (shape === 'ellipse' || shape === 'rectangle')
      spec.aspect = randRange(rand, 0.6, 0.85);
  }
  if (aligned) Object.assign(spec, nominalSpec(shape));


  if (defective) {
    const count = rand() < singleBias ? 1 : 1 + Math.floor(rand() * 3);
    for (let i = 0; i < count; i += 1) {
      spec.spikes.push(
        extended
          ? {
              kind: rand() < 0.3 ? 'blob' : 'spike',
              angle: randRange(rand, 0, TWO_PI),
              len: randRange(rand, 0.04, 0.14),
              wid: randRange(rand, 0.025, 0.08),
            }
          : {
              kind: 'spike',
              angle: randRange(rand, 0, TWO_PI),
              len: randRange(rand, 0.06, 0.13),
              wid: randRange(rand, 0.04, 0.08),
            },
      );
    }
  }
  return spec;
}


function drawBase(ctx, spec, size) {
  const cx = spec.cx * size;
  const cy = spec.cy * size;
  const r = spec.r * size;
  ctx.beginPath();
  switch (spec.type) {
    case 'circle':
      ctx.arc(cx, cy, r, 0, TWO_PI);
      break;
    case 'ellipse':
      ctx.ellipse(cx, cy, r, r * spec.aspect, spec.rot, 0, TWO_PI);
      break;
    case 'square':
```

```
    case 'rectangle':
      ctx.save();
      ctx.translate(cx, cy);
      ctx.rotate(spec.rot);
      ctx.rect(-r, -r * spec.aspect, r * 2, r * spec.aspect * 2);
      ctx.restore();
      break;
    default: {
      const n = POLYGON_SIDES[spec.type];
      const R = r * POLYGON_SCALE[spec.type];
      for (let k = 0; k < n; k += 1) {
        const angle = spec.rot + (TWO_PI * k) / n;
        const x = cx + R * Math.cos(angle);
        const y = cy + R * Math.sin(angle);
        if (k === 0) ctx.moveTo(x, y);
        else ctx.lineTo(x, y);
      }
      ctx.closePath();
    }
  }
  ctx.fill();
}

export function drawSpec(
  ctx,
  spec,
  size,
  { fg = '#ffffff', bg = '#000000' } = {},
) {
  ctx.fillStyle = bg;
  ctx.fillRect(0, 0, size, size);
  ctx.fillStyle = fg;
  drawBase(ctx, spec, size);

  const cx = spec.cx * size;
  const cy = spec.cy * size;
  for (const spike of spec.spikes) {
    const start = boundaryDistance(spec, spike.angle) * SPIKE_START * size;
    const end = spikeEnd(spec, spike) * size;
    const wid = spike.wid * size;
    ctx.save();
    ctx.translate(cx, cy);
    ctx.rotate(spike.angle);
    if (spike.kind === 'blob') {
      const neckEnd = start + (end - start) * 0.6;
      ctx.fillRect(start, -wid * 0.25, neckEnd - start, wid * 0.5);
      ctx.beginPath();
      ctx.arc(neckEnd, 0, wid * 0.8, 0, TWO_PI);
      ctx.fill();
    } else {
      ctx.fillRect(start, -wid / 2, end - start, wid);
    }
    ctx.restore();
  }
}
```

A.4. src/shared/lib/ml/histogram.js

```
export const BINS = 32;

export function normalizePolarity(gray, width, height) {
  let sum = 0;
  let count = 0;
  for (let x = 0; x < width; x += 1) {
    sum += gray[x] + gray[(height - 1) * width + x];
    count += 2;
  }
  for (let y = 1; y < height - 1; y += 1) {
    sum += gray[y * width] + gray[y * width + width - 1];
```

```
    count += 2;
  }
  if (sum / count <= 127) return gray;
  const out = new Uint8ClampedArray(gray.length);
  for (let i = 0; i < gray.length; i += 1) out[i] = 255 - gray[i];
  return out;
}

export function rgbaToGray(rgba) {
  const gray = new Uint8ClampedArray(rgba.length / 4);
  for (let i = 0, j = 0; i < rgba.length; i += 4, j += 1) {
    gray[j] = Math.round(
      0.299 * rgba[i] + 0.587 * rgba[i + 1] + 0.114 * rgba[i + 2],
    );
  }
  return gray;
}

export function histogram(gray, bins = BINS) {
  const counts = new Array(bins).fill(0);
  const step = 256 / bins;
  for (let i = 0; i < gray.length; i += 1)
    counts[Math.min(bins - 1, Math.floor(gray[i] / step))] += 1;
  return counts;
}

export function cumulativeHistogram(gray, bins = BINS) {
  const counts = histogram(gray, bins);
  const total = gray.length;
  const out = new Array(bins);
  let acc = 0;
  for (let k = 0; k < bins; k += 1) {
    acc += counts[k];
    out[k] = acc / total;
  }
  return out;
}
```

A.5. src/shared/lib/ml/diff.js

```
import { histogram } from './histogram.js';

export function differenceImages(gray, reference) {
  const excess = new Uint8ClampedArray(gray.length);
  const deficit = new Uint8ClampedArray(gray.length);
  for (let i = 0; i < gray.length; i += 1) {
    const d = gray[i] - reference[i];
    if (d > 0) excess[i] = d;
    else deficit[i] = -d;
  }
  return { excess, deficit };
}

export const absoluteDifference = (gray, reference) => {
  const out = new Uint8ClampedArray(gray.length);
  for (let i = 0; i < gray.length; i += 1)
    out[i] = Math.abs(gray[i] - reference[i]);
  return out;
};

function cumulativeOfNonZero(diff, bins) {
  const counts = histogram(diff, bins);
  const out = [];
  let acc = 0;
  for (let k = 1; k < bins; k += 1) {
    acc += counts[k];
    out.push(acc / diff.length);
```

```
  }
  return out;
}

export function diffHistogramFeatures(gray, reference, bins) {
  const { excess, deficit } = differenceImages(gray, reference);
  return [
    ...cumulativeOfNonZero(excess, bins),
    ...cumulativeOfNonZero(deficit, bins),
  ];
}

export const diffBinsFromLength = (length) => length / 2 + 1;
```

A.6. src/shared/lib/ml/pipeline.js

```
import { FILE_FORMATS, MIXED_FORMAT, TRAIN_SIZE } from '@/shared/config/ml.js';
import { encodeBmp } from '@/shared/lib/image/bmp.js';
import { absoluteDifference, diffHistogramFeatures } from './diff.js';
import {
  cumulativeHistogram,
  normalizePolarity,
  rgbaToGray,
} from './histogram.js';
import { createRng } from './rng.js';
import { drawSpec, nominalSpec, randomSpec } from './shapes.js';

export function downscaleToGray(source) {
  const canvas = new OffscreenCanvas(TRAIN_SIZE, TRAIN_SIZE);
  const ctx = canvas.getContext('2d', { willReadFrequently: true });
  ctx.imageSmoothingEnabled = true;
  ctx.imageSmoothingQuality = 'high';
  ctx.drawImage(source, 0, 0, TRAIN_SIZE, TRAIN_SIZE);
  const rgba = ctx.getImageData(0, 0, TRAIN_SIZE, TRAIN_SIZE).data;
  return normalizePolarity(rgbaToGray(rgba), TRAIN_SIZE, TRAIN_SIZE);
}

export async function decodeToGray(blob) {
  const bitmap = await createImageBitmap(blob);
  const { width, height } = bitmap;
  const gray = downscaleToGray(bitmap);
  bitmap.close();
  return { gray, width, height };
}

const referenceCache = new Map();
export function referenceGray(type) {
  if (!referenceCache.has(type)) {
    const size = 256;
    const canvas = new OffscreenCanvas(size, size);
    drawSpec(canvas.getContext('2d'), nominalSpec(type), size);
    referenceCache.set(type, downscaleToGray(canvas));
  }
  return referenceCache.get(type);
}

export function computeFeatures(gray, { refType, bins }) {
  return diffHistogramFeatures(gray, referenceGray(refType), bins);
}

export function displayCumulative(gray, { refType, bins }) {
  return cumulativeHistogram(
    absoluteDifference(gray, referenceGray(refType)),
    bins,
  );
}
```

```
function encodeCanvas(canvas, format) {
  if (format === 'bmp') {
    const ctx = canvas.getContext('2d');
    return encodeBmp(ctx.getImageData(0, 0, canvas.width, canvas.height));
  }
  return canvas.convertToBlob({
    type: format === 'jpg' ? 'image/jpeg' : 'image/png',
    quality: 0.9,
  });
}


export async function prepareSample({
  id,
  index,
  group,
  format,
  variety,
  shapes,
  bins,
  singleBias,
  seed,
}) {
  const rand = createRng(seed);
  const label = index % 2 === 0 ? 'normal' : 'defect';
  const spec = randomSpec(rand, {
    defective: label === 'defect',
    variety,
    shapes,
    singleBias,
    aligned: true,
  });


  const canvas = new OffscreenCanvas(group.size, group.size);
  drawSpec(canvas.getContext('2d'), spec, group.size);


  const fileFormat =
    format === MIXED_FORMAT
      ? FILE_FORMATS[id % FILE_FORMATS.length].value
      : format;
  const blob = await encodeCanvas(canvas, fileFormat);
  const { gray } = await decodeToGray(blob);


  return {
    id,
    label,
    group: group.id,
    format: fileFormat,
    bytes: blob.size,
    features: computeFeatures(gray, { refType: shapes[0], bins }),
  };
}
```

A.7. src/shared/lib/ml/dataset.js

```
import { createRng } from './rng.js';


export function fitNormalizer(samples) {
  const n = samples[0].features.length;
  const min = new Array(n).fill(Infinity);
  const max = new Array(n).fill(-Infinity);
  for (const { features } of samples) {
    features.forEach((v, i) => {
      if (v < min[i]) min[i] = v;
      if (v > max[i]) max[i] = v;
    });
  }
```

```
  return { min, max };
}

export function applyNormalizer({ min, max }, features) {
  return features.map((v, i) => {
    const span = max[i] - min[i];
    if (span < 1e-9) return 0;
    return Math.min(1, Math.max(0, (v - min[i]) / span));
  });
}

export function splitSamples(samples, trainRatio = 0.8, seed = 42) {
  const rand = createRng(seed);
  const shuffled = samples.slice();
  for (let i = shuffled.length - 1; i > 0; i -= 1) {
    const j = Math.floor(rand() * (i + 1));
    [shuffled[i], shuffled[j]] = [shuffled[j], shuffled[i]];
  }
  const cut = Math.floor(shuffled.length * trainRatio);
  return { train: shuffled.slice(0, cut), test: shuffled.slice(cut) };
}
```

A.8. src/shared/lib/ml/tasks.js

```
import { DATASET_GROUPS } from '@/shared/config/ml.js';

export function buildTasks({
  groups,
  format,
  imagesPerGroup,
  variety = 'basic',
  shapes,
  bins = 32,
  singleBias = 0,
  seed = 1000,
}) {
  const tasks = [];
  for (const group of DATASET_GROUPS) {
    if (!groups.includes(group.id)) continue;
    for (let index = 0; index < imagesPerGroup; index += 1) {
      const id = tasks.length;
      tasks.push({
        id,
        index,
        group,
        format,
        variety,
        shapes,
        bins,
        singleBias,
        seed: seed + id * 7919,
      });
    }
  }
  return tasks;
}
```

A.9. src/shared/api/ml/index.js

```
export { prepareDataset, defaultWorkers, maxWorkers } from './pool.js';

let worker = null;
let seq = 0;
const pending = new Map();

function getWorker() {
  if (!worker) {
    worker = new Worker(new URL('./train.worker.js', import.meta.url), {
```

```
      type: 'module',
    });
    worker.onmessage = ({ data }) => {
      const entry = pending.get(data.id);
      if (!entry) return;
      if (data.type === 'progress') entry.onProgress?.(data);
      else {
        pending.delete(data.id);
        if (data.type === 'done') entry.resolve(data.result);
        else entry.reject(new Error(data.message));
      }
    };
    worker.onerror = (event) => {
      pending.forEach((entry) =>
        entry.reject(new Error(event.message || 'Помилка воркера')),
      );
      pending.clear();
    };
  }
  return worker;
}

function request(type, payload, onProgress) {
  return new Promise((resolve, reject) => {
    seq += 1;
    pending.set(seq, { resolve, reject, onProgress });
    getWorker().postMessage({ id: seq, type, payload });
  });
}

let loadedModelId = null;

export const markModelLoaded = (id) => {
  loadedModelId = id;
};

export const trainInWorker = (samples, config, onProgress) => {
  loadedModelId = null;
  return request('train', { samples, config }, onProgress);
};

export async function recognizeInWorker(blob, model, onProgress) {
  const payload = { blob };
  if (loadedModelId !== model.id) {
    const stored = model.getPayload();
    if (!stored)
      throw new Error(
        'Дані моделі не знайдено у сховищі браузера. Навчіть модель заново.',
      );
    payload.model = stored;
  }
  const result = await request('recognize', payload, onProgress);
  loadedModelId = model.id;
  return result;
}
```

A.10. src/shared/api/ml/pool.js

```
import { MAX_WORKERS } from '@/shared/config/ml.js';
import { prepareSample } from '@/shared/lib/ml/pipeline.js';

const CHUNK_SIZE = 6;

let pool = [];
let seq = 0;
const pending = new Map();
```

```
export const defaultWorkers = () =>
  Math.max(1, Math.min(navigator.hardwareConcurrency || 4, MAX_WORKERS));
export const maxWorkers = () => Math.max(1, navigator.hardwareConcurrency || 4);


function spawn() {
  const worker = new Worker(new URL('./prep.worker.js', import.meta.url), {
    type: 'module',
  });
  worker.onmessage = ({ data }) => {
    const entry = pending.get(data.id);
    if (!entry) return;
    pending.delete(data.id);
    if (data.error) entry.reject(new Error(data.error));
    else entry.resolve(data.results);
  };
  worker.onerror = (event) => {
    pending.forEach((entry, id) => {
      if (entry.worker === worker) {
        pending.delete(id);
        entry.reject(
          new Error(event.message || 'Помилка воркера підготовки даних'),
        );
      }
    });
  };
  return worker;
}


function call(worker, message) {
  return new Promise((resolve, reject) => {
    seq += 1;
    pending.set(seq, { resolve, reject, worker });
    worker.postMessage({ id: seq, ...message });
  });
}


export async function ensurePool(size) {
  while (pool.length < size) pool.push(spawn());
  await Promise.all(
    pool.slice(0, size).map((worker) => call(worker, { type: 'ping' })),
  );
}


const yieldToUi = () => new Promise((resolve) => setTimeout(resolve, 0));


function chunked(items, size) {
  const out = [];
  for (let i = 0; i < items.length; i += size) out.push(items.slice(i, i + size));
  return out;
}


export async function prepareDataset(tasks, { workers, onProgress }) {
  const results = [];
  let done = 0;
  const report = (count) => {
    done += count;
    onProgress?.(done, tasks.length);
  };


  if (workers === 0) {
    for (const part of chunked(tasks, CHUNK_SIZE)) {
      for (const task of part) results.push(await prepareSample(task));
      report(part.length);
      await yieldToUi();
    }
```

```
  } else {
    await ensurePool(workers);
    const queue = chunked(tasks, CHUNK_SIZE);
    let next = 0;
    await Promise.all(
      pool.slice(0, workers).map(async (worker) => {
        while (next < queue.length) {
          const part = queue[next];
          next += 1;
          results.push(...(await call(worker, { type: 'prepare', tasks: part })));
          report(part.length);
        }
      }),
    );
  }
  return results.sort((a, b) => a.id - b.id);
}
```

A.11. src/shared/api/ml/prep.worker.js

```
import { prepareSample } from '@/shared/lib/ml/pipeline.js';

self.onmessage = async ({ data }) => {
  const { id, type, tasks } = data;
  try {
    if (type === 'ping') {
      self.postMessage({ id, results: [] });
      return;
    }
    const results = [];
    for (const task of tasks) results.push(await prepareSample(task));
    self.postMessage({ id, results });
  } catch (error) {
    self.postMessage({ id, error: error.message });
  }
};
```

A.12. src/shared/api/ml/train.worker.js

```
import brain from 'brain.js/dist/neural-network.js';
import { CLASSES } from '@/shared/lib/ml/shapes.js';
import {
  applyNormalizer,
  fitNormalizer,
  splitSamples,
} from '@/shared/lib/ml/dataset.js';
import { FEATURE_MODE, RECOGNITION_STEPS } from '@/shared/config/ml.js';

const [STEP_DECODE, STEP_FEATURES, STEP_CLASSIFY] = RECOGNITION_STEPS;
import { diffBinsFromLength } from '@/shared/lib/ml/diff.js';
import {
  computeFeatures,
  decodeToGray,
  displayCumulative,
} from '@/shared/lib/ml/pipeline.js';

const NeuralNetwork = brain.NeuralNetwork ?? brain.default ?? brain;

let net = null;
let normalizer = null;

const reply = (id, type, data) => self.postMessage({ id, type, ...data });

function argmax(scores) {
  return CLASSES.reduce(
    (best, name) => (scores[name] > scores[best] ? name : best),
    CLASSES[0],
```

```
  );
}

function tally(table, key, ok) {
  table[key] ??= { correct: 0, total: 0 };
  table[key].total += 1;
  if (ok) table[key].correct += 1;
}

function train(id, { samples, config }) {
  const started = performance.now();
  const { train: trainSet, test: testSet } = splitSamples(samples);
  normalizer = {
    ...fitNormalizer(trainSet),
    mode: FEATURE_MODE,
    refType: config.shapes[0],
  };

  const toData = (s) => ({
    input: applyNormalizer(normalizer, s.features),
    output: { [s.label]: 1 },
  });
  net = new NeuralNetwork({
    hiddenLayers: config.hiddenNeurons > 0 ? [config.hiddenNeurons] : [],
  });

  const stats = net.train(trainSet.map(toData), {
    iterations: config.epochs,
    errorThresh: 0.005,
    learningRate: config.learningRate,
    callbackPeriod: Math.max(1, Math.floor(config.epochs / 100)),
    callback: ({ iterations: epoch, error }) =>
      reply(id, 'progress', { epoch, epochs: config.epochs, error }),
  });

  const confusion = {
    normal: { normal: 0, defect: 0 },
    defect: { normal: 0, defect: 0 },
  };
  const byGroup = {};
  const byFormat = {};
  let correct = 0;
  for (const sample of testSet) {
    const predicted = argmax(
      net.run(applyNormalizer(normalizer, sample.features)),
    );
    const ok = predicted === sample.label;
    confusion[sample.label][predicted] += 1;
    tally(byGroup, sample.group, ok);
    tally(byFormat, sample.format, ok);
    if (ok) correct += 1;
  }

  reply(id, 'done', {
    result: {
      accuracy: correct / testSet.length,
      confusion,
      byGroup,
      byFormat,
      trainError: stats.error,
      epochs: stats.iterations,
      trainSize: trainSet.length,
      testSize: testSet.length,
      trainMs: performance.now() - started,
      netJson: net.toJSON(),
      normalizer,
    },
```

```
  });
}

async function recognize(id, { blob, model }) {
  if (model) {
    net = new NeuralNetwork();
    net.fromJSON(model.netJson);
    normalizer = model.normalizer;
  }
  if (!net) throw new Error('Модель не завантажена. Навчіть або оберіть модель.');
  const total = performance.now();
  const steps = {};
  let mark = total;
  const lap = (name) => {
    const now = performance.now();
    steps[name] = now - mark;
    mark = now;
  };

  reply(id, 'progress', STEP_DECODE);
  const { gray, width, height } = await decodeToGray(blob);
  lap('decode');

  reply(id, 'progress', STEP_FEATURES);
  if (normalizer.mode !== FEATURE_MODE) {
    throw new Error(
      'Модель створена старою версією застосунку. Навчіть її заново.',
    );
  }
  const bins = diffBinsFromLength(normalizer.min.length);
  const { refType } = normalizer;
  const features = computeFeatures(gray, { refType, bins });
  const cumulative = displayCumulative(gray, { refType, bins });
  lap('histogram');

  reply(id, 'progress', STEP_CLASSIFY);
  const scores = net.run(applyNormalizer(normalizer, features));
  const label = argmax(scores);
  lap('predict');

  reply(id, 'done', {
    result: {
      label,
      scores: { normal: scores.normal, defect: scores.defect },
      cumulative,
      width,
      height,
      steps,
      totalMs: performance.now() - total,
    },
  });
}

self.onmessage = async ({ data }) => {
  const { id, type, payload } = data;
  try {
    if (type === 'train') train(id, payload);
    else if (type === 'recognize') await recognize(id, payload);
  } catch (error) {
    reply(id, 'error', { message: error.message });
  }
};
```

A.13. src/features/train-model/model/trainModel.js

```
import { logAdded } from '@/entities/log';
import {
```

```
  modelAdded,
  prepProgressed,
  trainStarted,
  trainingFailed,
  trainingFinished,
  trainingProgressed,
  trainingStarted,
} from '@/entities/model';
import { markModelLoaded, prepareDataset, trainInWorker } from '@/shared/api/ml';
import { DATASET_GROUPS } from '@/shared/config/ml.js';
import { formatBytes, formatMs } from '@/shared/lib/format.js';
import { SHAPE_LABELS } from '@/shared/lib/ml/shapes.js';
import { buildTasks } from '@/shared/lib/ml/tasks.js';
import { savePayload } from '@/shared/lib/storage/models.js';


export const describeMode = (workers) =>
  workers === 0
    ? 'головний потік (без розпаралелення)'
    : `Web Workers × ${workers}`;


export const trainModel = () => async (dispatch, getState) => {
  const { config } = getState().model;
  const tasks = buildTasks(config);
  const workers = config.mode === 'main' ? 0 : config.workers;
  const groups = DATASET_GROUPS.filter((g) => config.groups.includes(g.id))
    .map((g) => `${g.label} ${g.size}×${g.size}`)
    .join(', ');
  const log = (message, level) => dispatch(logAdded(message, level));


  const started = performance.now();
  dispatch(trainingStarted({ startedAt: Date.now(), total: tasks.length }));
  log(
    `Старт навчання: ${tasks.length} зображень (${groups}), формат: ${config.format},
фігури: ${config.shapes.join(', ')}, варіативність: ${config.variety}`,
  );
  log(
    `Підготовка даних: ${describeMode(workers)}. Епох: ${config.epochs}, швидкість
навчання: ${config.learningRate}, нейронів: ${config.hiddenNeurons}`,
  );


  try {
    let nextLogAt = 0.25;
    const samples = await prepareDataset(tasks, {
      workers,
      onProgress: (done, total) => {
        dispatch(prepProgressed({ done, total }));
        if (done / total >= nextLogAt && done < total) {
          log(`Оброблено ${done}/${total} зображень`);
          nextLogAt += 0.25;
        }
      },
    });
    const prepMs = performance.now() - started;
    const bytes = samples.reduce((sum, s) => sum + s.bytes, 0);
    log(
      `Підготовку завершено за ${formatMs(prepMs)}: ${samples.length} зображень,
${formatBytes(bytes)} у файлах`,
      'success',
    );


    dispatch(trainStarted());
    let lastQuarter = 0;
    const metrics = await trainInWorker(
      samples,
      config,
      ({ epoch, epochs, error }) => {
        dispatch(trainingProgressed({ epoch, epochs, error }));
```

```
        const quarter = Math.floor((epoch / epochs) * 4);
        if (quarter > lastQuarter) {
          lastQuarter = quarter;
          log(`Епоха ${epoch}/${epochs}, помилка ${error.toFixed(4)}`);
        }
      },
    );

    const totalMs = performance.now() - started;
    const { netJson, normalizer, ...rest } = metrics;
    const fullMetrics = {
      ...rest,
      prepMs,
      totalMs,
      imageCount: samples.length,
      workers,
    };
    dispatch(trainingFinished(fullMetrics));
    log(
      `Навчання завершено: ${metrics.epochs} епох за ${formatMs(metrics.trainMs)},
точність на тесті ${(metrics.accuracy * 100).toFixed(1)}%, разом ${formatMs(totalMs)}`,
      'success',
    );

    const id = crypto.randomUUID();
    const shapeNames = config.shapes.map((s) => SHAPE_LABELS[s]).join(', ');
    const name = `Модель ${getState().model.seq + 1} · ${shapeNames}`;
    try {
      savePayload(id, { netJson, normalizer });
      dispatch(
        modelAdded({
          id,
          name,
          createdAt: new Date().toISOString(),
          config: { ...config },
          metrics: fullMetrics,
        }),
      );
      markModelLoaded(id);
      log(
        `Модель «${name}» збережено у браузері й обрано для розпізнавання`,
        'success',
      );
    } catch (error) {
      log(
        `Модель не збережено (сховище браузера недоступне або заповнене):
${error.message}`,
        'warn',
      );
    }
  } catch (error) {
    dispatch(trainingFailed(error.message));
    log(`Помилка навчання: ${error.message}`, 'error');
  }
};
```

A.14. src/features/recognize-image/model/recognizeImage.js

```
import { logAdded } from '@/entities/log';
import { selectActiveModel } from '@/entities/model';
import {
  recognitionFailed,
  recognitionFinished,
  recognitionProgressed,
  recognitionStarted,
} from '@/entities/recognition';
import { recognizeInWorker } from '@/shared/api/ml';
import { formatBytes, formatMs, formatPercent } from '@/shared/lib/format.js';
import { getCurrentFile } from '@/shared/lib/image/fileStore.js';
```

```
import { createThumbnail } from '@/shared/lib/image/thumbnail.js';
import { loadPayload } from '@/shared/lib/storage/models.js';


export const recognizeImage = () => async (dispatch, getState) => {
  const file = getCurrentFile();
  const { meta } = getState().image;
  const model = selectActiveModel(getState());
  if (!file || !meta || !model) return;


  dispatch(recognitionStarted());
  dispatch(
    logAdded(
      `Розпізнавання: ${meta.name} (${formatBytes(meta.size)},
${meta.width}×${meta.height}), модель «${model.name}»`,
    ),
  );
  try {
    const result = await recognizeInWorker(
      file,
      { id: model.id, getPayload: () => loadPayload(model.id) },
      (p) => dispatch(recognitionProgressed(p)),
    );
    const url = URL.createObjectURL(file);
    const thumbUrl = await createThumbnail(file).catch(() => url);
    dispatch(
      recognitionFinished({
        result: { ...result, modelName: model.name },
        image: { name: meta.name, size: meta.size, url, thumbUrl },
      }),
    );
    dispatch(
      logAdded(
        `Результат: ${result.label === 'defect' ? 'дефект' : 'без дефекту'}
(${formatPercent(result.scores[result.label])}), час ${formatMs(result.totalMs)}`,
        result.label === 'defect' ? 'warn' : 'success',
      ),
    );
  } catch (error) {
    dispatch(recognitionFailed(error.message));
    dispatch(logAdded(`Помилка розпізнавання: ${error.message}`, 'error'));
  }
};
```